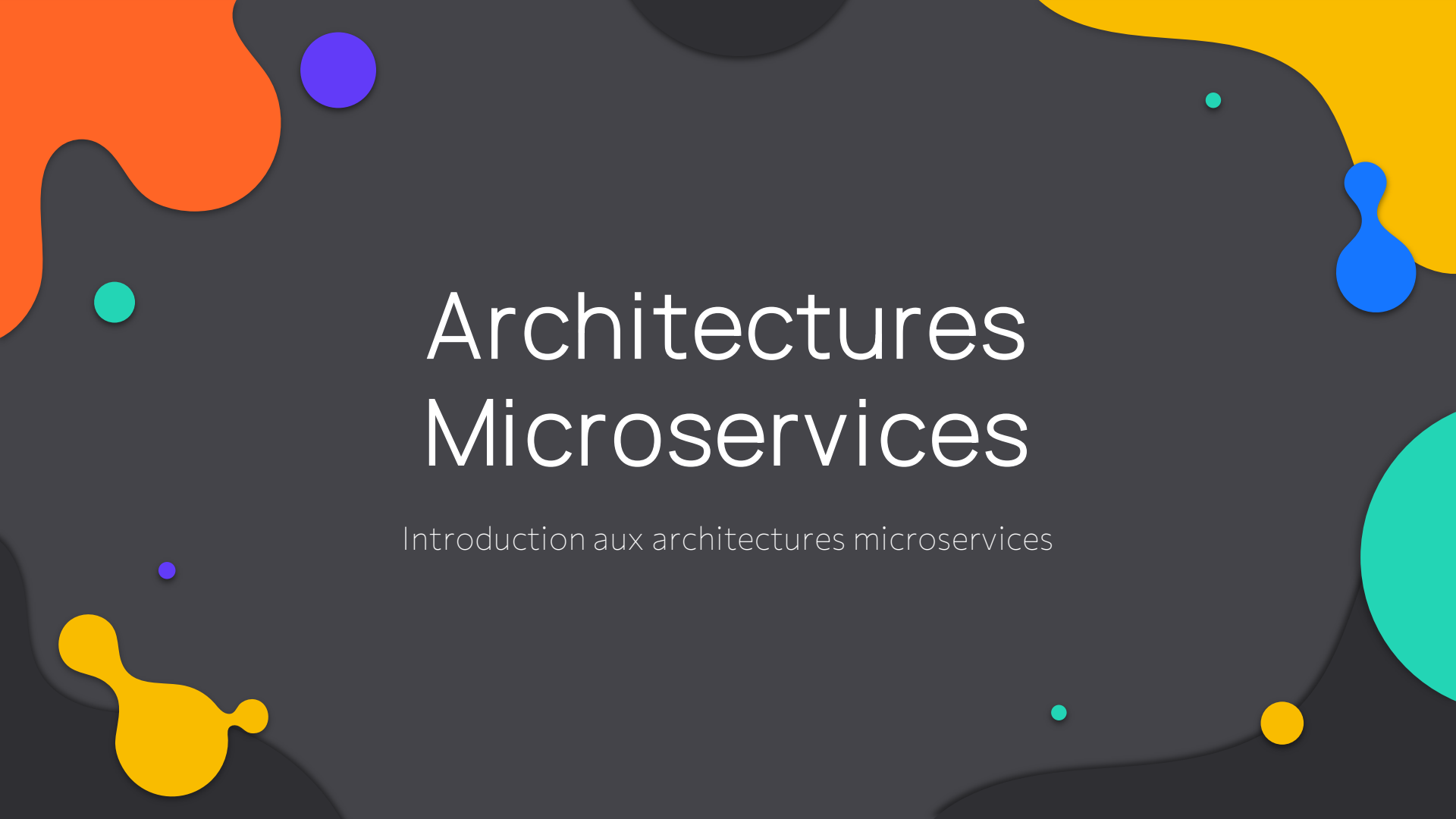




Architectures Microservices

Introduction aux architectures microservices

Objectifs du module



Compétences

Acquérir une solide compréhension des fondamentaux de l'architecture en microservices et savoir l'appliquer en cas réel.

Notation du module



Contrôle Continu

Devoir surveillé comprenant des questions à choix multiples, des questions ouvertes, et des mises en situation réelles.



Projet

Initiative de projet visant à consolider et à enrichir vos compétences en matière d'architecture microservices.

Définition

Style architectural dans lequel une application est divisée en un ensemble de petits services.

Chaque service exécute un processus unique qui communique avec les autres à travers des interfaces bien définies, généralement des API HTTP.

Chaque microservice est autonome, responsable d'une seule partie de la fonctionnalité de l'entreprise et peut être développé, déployé, opéré et mis à l'échelle indépendamment des autres services.

Principes clés

Indépendant

Chaque service fonctionne de manière indépendante des autres.

Modulaire

Facilite la compréhension, le développement et le test de l'application.

Décentralisé

Favorise la décentralisation de la gestion des données et des processus.

Déployable

Permet de déployer des services de manière indépendante, essentiel pour les méthodologies agiles.

Scalable

Chaque service peut être mis à l'échelle indépendamment en fonction des demandes.

Echec isolé

Les défaillances dans un service spécifique affectent moins l'ensemble du système.

Architecture Monolithique

Un monolithe est une architecture de logiciel traditionnelle où toutes les fonctions d'une application sont étroitement liées et opèrent sous une seule et même unité de déploiement.

Ce type d'architecture est souvent caractérisé par :

- Couplage fort : *Les composants de l'application sont interdépendants, rendant les mises à jour et la maintenance complexes.*
- Déploiement unique : *Tout changement, même mineur, nécessite le redéploiement de l'ensemble de l'application.*
- Scalabilité limitée : *Toute l'application doit être mise à l'échelle ensemble plutôt que de manière indépendante pour ses différentes fonctions.*

Monolithique VS Microservices



Agilité

Rapidité de développement et de déploiement



Technologie hétérogène

Utilisation de plusieurs technologies au sein d'un projet



Résilience

Isolement des fonctionnalités sans impact sur l'application



Complexité de gestion

Complexité supplémentaire en terme de gestion des services, de communication entre eux et de surveillance.

Histoire et Evolution

Les architectures en microservices, bien que non nouvelles, se sont popularisées avec le cloud, les conteneurs et le DevOps.

Historiquement monolithiques, elles ont évolué vers les microservices pour répondre à des besoins croissants de scalabilité et d'agilité, une transition adoptée précocement par des géants technologiques comme Amazon, Netflix, et Google pour gérer de grands volumes de données et des demandes client rapidement changeantes.

Découpage en microservices

C'est une étape critique dans la conception d'une architecture basée sur les microservices. Cela implique de décomposer une application en petites parties qui peuvent être développées, déployées et gérées de manière indépendante.

Il existe différentes stratégies pour permettre ce découpage :

- **Domain-Driven Design (DDD)** : *Définition des limites autour des domaines métier distincts de l'entreprise. Chaque microservice gère une partie spécifique du modèle de domaine, ce qui permet une séparation claire et logique des responsabilités.*
- **Bounded Contexts** : *Ils aident à définir où commencent et finissent les responsabilités pour un aspect spécifique du domaine. Cela simplifie la communication entre les équipes et évite les dépendances inutiles entre les services.*
- **Single Responsibility Principle** : *Chaque microservice doit être responsable d'une seule fonctionnalité ou d'un seul aspect du système, ce qui contribue à minimiser l'interdépendance entre les services.*

Communication entre les services



API REST

Permet de demander des données ou des actions de la part d'autres services.



Protocoles de message

Divers protocoles peuvent être utilisés pour une communication plus efficace et structurée.



Events Async

Des outils permettent aux services de publier des événements qui peuvent être consommés par un ou plusieurs services.

Gestion des données

● DB par service

Chaque microservice gère sa propre base de données, ce qui évite les conflits de données et réduit les dépendances.

Synchronisation

Parfois, il peut être nécessaire de répliquer les données entre les services. Il existe des techniques et des outils de synchronisation qui permettent de s'assurer que les données sont à jour et cohérentes à travers les services.

Consistance

Les modèles de transaction compensatoires, souvent implémentés à travers des patterns comme le Saga, permettent de gérer la cohérence sans verrouillage centralisé.



Frameworks et Langages



Spring Boot (Java)

Permet de créer facilement des microservices autonomes et prêts pour la production.

Django/Flask (Python)

Django est utilisé pour des applications plus lourdes avec beaucoup de fonctionnalités intégrées tandis que Flask est plus léger et flexible.

Express (Node.js)

Utilisé pour construire des applications web et des API.

Go

Son point fort et sa performance et sa facilité de déploiement, il est devenu un choix populaire pour les microservices.





Conteneurisation et Orchestration



Docker

C'est la référence de conteneurisation, permettant de packager et d'exécuter des applications dans des conteneurs légers et isolés.

Kubernetes

Systeme d'orchestration de conteneurs qui gère l'automatisation du déploiement, du scaling et des opérations des applications conteneurisés.





API Gateway & Service Mesh

- API Gateway :

L'API Gateway agit comme un point d'entrée unique pour toutes les requêtes externes à l'application microservices.



Il peut gérer l'authentification, la surveillance, le load balancing, le caching, la gestion des politiques de sécurité, et la mise en œuvre des quotas et des contrôles d'accès.

- Service Mesh :

Un Service Mesh aide à gérer la communication complexe entre les services, y compris le routage des requêtes, le monitoring, et la sécurisation des communications de service à service.

Il implémente des fonctionnalités avancées telles que les retry policies, les failovers, les observations, et la sécurité des transports.





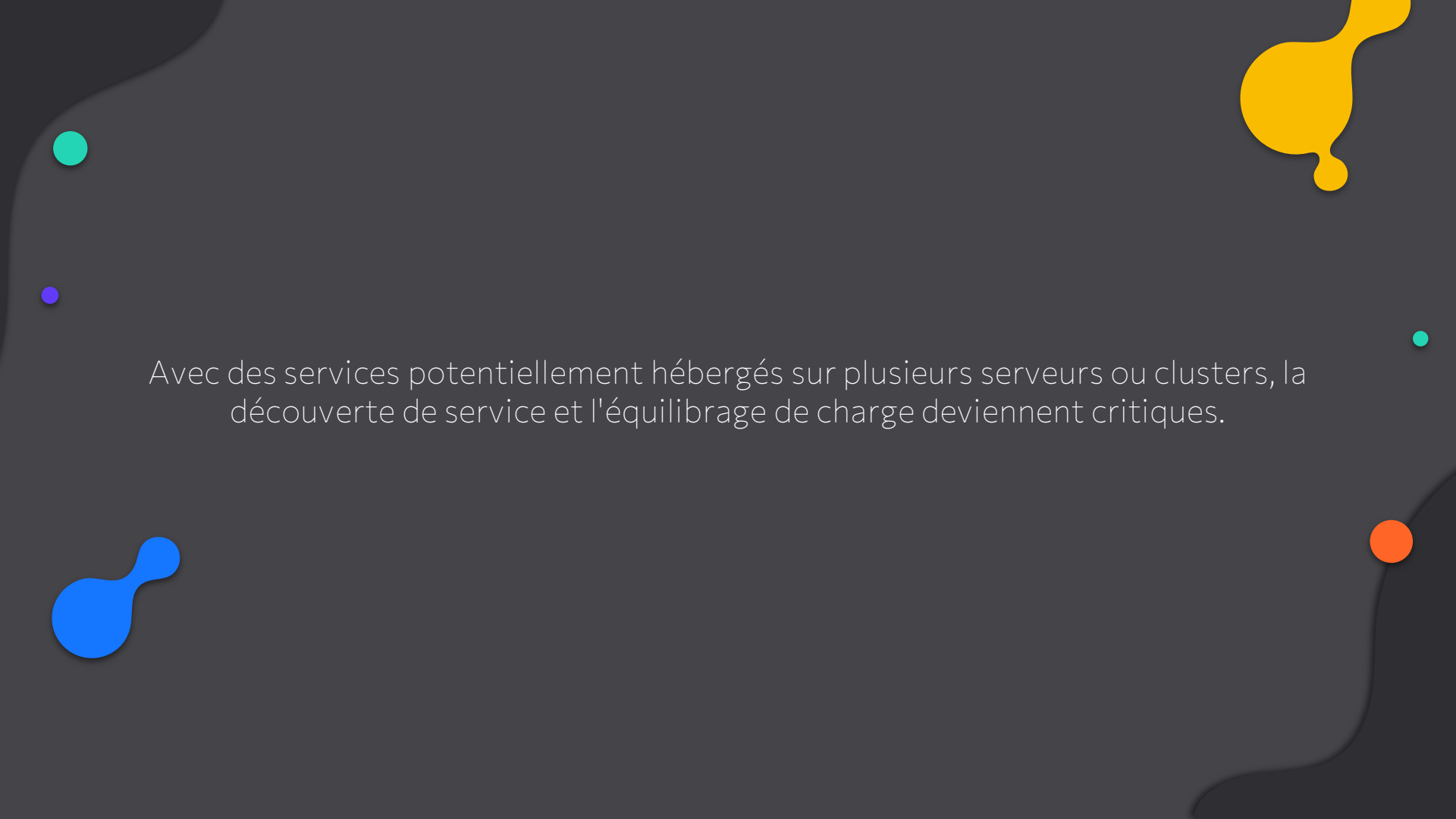
Le déploiement efficace de microservices nécessite une approche robuste de CI/CD pour gérer les multiples services qui peuvent être développés et déployés indépendamment.

Défis

- Coordonner les déploiements fréquents et indépendants de divers microservices peut être complexe.
- S'assurer que les services sont déployés sans interruption de service nécessite une planification minutieuse.

Solutions

- Automatisation des tests :
Automatiser les tests unitaires, d'intégration et de charge pour s'assurer que le code fonctionne correctement avant le déploiement.
- Pipelines de déploiement :
Utiliser des outils comme Jenkins, GitLab CI, ou CircleCI pour automatiser les phases de build, test, et déploiement.
- Déploiements progressifs :
Techniques comme le blue-green deployment ou le canary release permettent de minimiser les risques en déployant progressivement la nouvelle version à un sous-ensemble d'utilisateurs avant le déploiement complet.



Avec des services potentiellement hébergés sur plusieurs serveurs ou clusters, la découverte de service et l'équilibrage de charge deviennent critiques.

Défis

Les microservices nécessitent un moyen dynamique de localiser et communiquer avec les autres services dans un environnement potentiellement changeant.

Solutions

- Découverte de service :

Utiliser des registres de services comme Eureka, Consul ou Zookeeper, qui permettent aux services de s'enregistrer lors de leur démarrage et d'être découverts par d'autres services.

- Équilibrage de charge :

Des outils comme Nginx ou HAProxy peuvent être utilisés pour distribuer les requêtes entrantes entre les instances de services, tandis que des solutions comme Istio ou Linkerd offrent des capacités d'équilibrage de charge sophistiquées à l'intérieur du cluster.



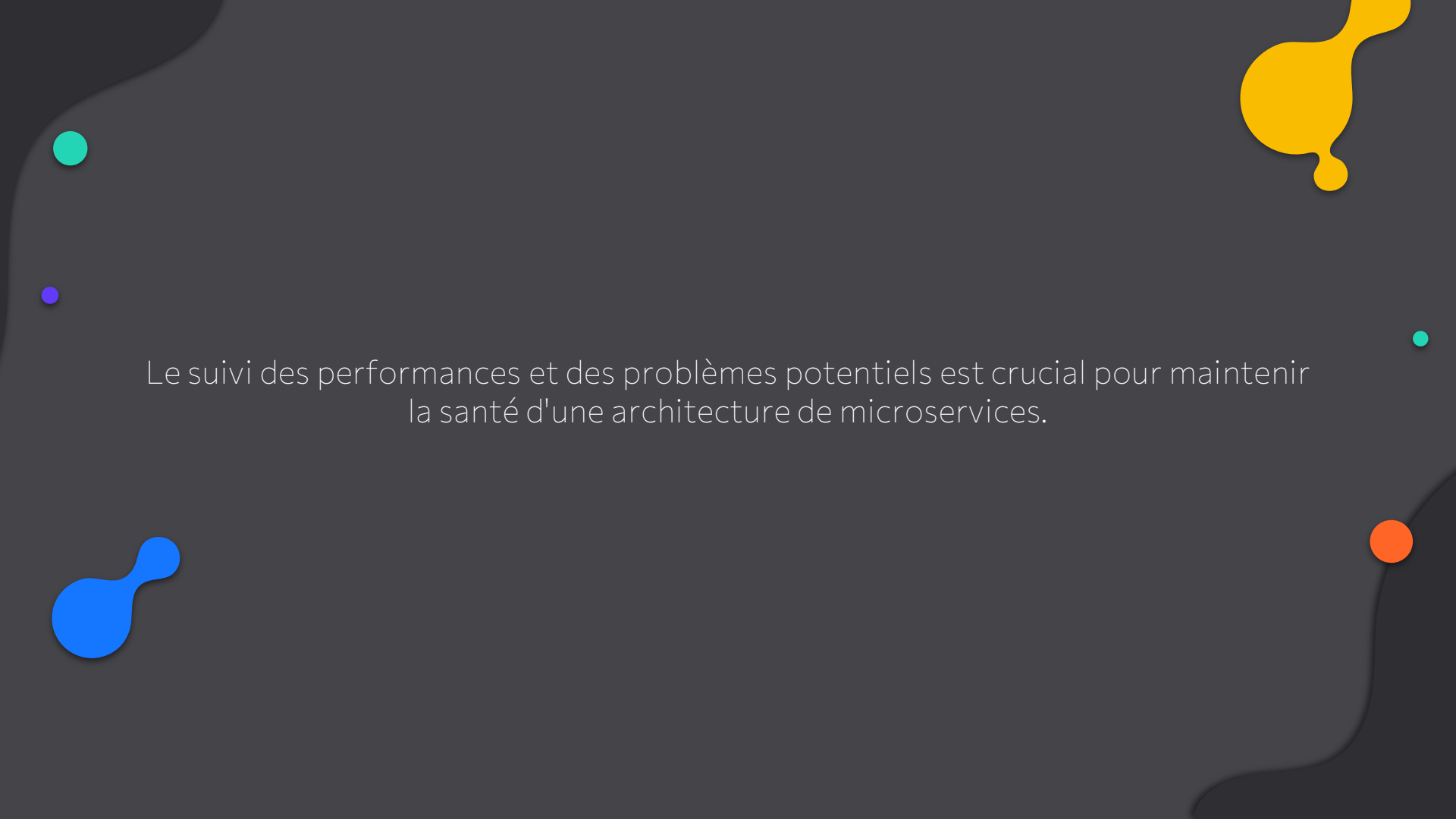
La sécurité est une préoccupation majeure dans les architectures distribuées où différents services peuvent avoir différents niveaux d'exposition et de vulnérabilité.

Défis

Assurer la sécurité des données transitant entre les services et authentifier les requêtes entre les services pour éviter les accès non autorisés.

Solutions

- **Authentification et autorisation :**
Implémenter des standards comme OAuth et JWT pour gérer l'authentification et les autorisations à travers les services.
- **Chiffrement :**
Utiliser TLS pour sécuriser les communications entre services pour éviter les écoutes indiscretes et les attaques de type man-in-the-middle.
- **API Gateways :**
Les API gateways peuvent également offrir des couches de sécurité supplémentaires, comme la validation des tokens, la limitation des taux, et le filtrage des requêtes.



Le suivi des performances et des problèmes potentiels est crucial pour maintenir la santé d'une architecture de microservices.

Défis

Suivre les interactions entre services multiples et distribués peut être complexe, et les problèmes peuvent ne pas être immédiatement apparents.

Solutions

- Centralisation des Logs :

Utiliser des outils comme ELK Stack (Elasticsearch, Logstash, Kibana) ou Splunk pour centraliser et analyser les logs provenant de divers services.

- Monitoring :

Implémenter des solutions de monitoring comme Prometheus pour collecter et stocker les métriques des services, et Grafana pour visualiser ces données.

- Tracing Distribué :

Utiliser des outils comme Jaeger ou Zipkin pour tracer les requêtes à travers les différents services, ce qui aide à diagnostiquer les problèmes de latence et d'échec dans les interactions de service.

Etude de cas : Netflix

Contexte:

Netflix est souvent cité comme un cas d'école pour l'adoption réussie de l'architecture en microservices.

Ayant commencé comme une simple plateforme de location de DVD par courrier, Netflix a migré vers la diffusion en continu en ligne, nécessitant une évolution majeure de leur architecture technologique pour gérer la croissance rapide du nombre d'utilisateurs et des demandes de contenu.

Etude de cas : Netflix

Problèmes rencontrés :

- Scalabilité:
L'architecture monolithique initiale de Netflix ne pouvait pas gérer efficacement les pics de demandes globales.
- Rapidité de déploiement :
Le développement de nouvelles fonctionnalités était lent, affectant la capacité de Netflix à innover rapidement.
- Résilience :
Les pannes de service étaient fréquentes et impactaient l'expérience utilisateur sur toute la plateforme.

Etude de cas : Netflix

Solution mise en œuvre :

- Découpage en microservices :
Netflix a progressivement refactorisé son application monolithique en microservices, chaque fonctionnalité devenant un service indépendant.
- Cloud et conteneurisation :
Adoption d'AWS pour le déploiement des microservices et utilisation de conteneurs pour améliorer la portabilité et l'efficacité.
- Déploiement continu :
Mise en place de pipelines de CI/CD permettant des mises à jour fréquentes et minimisant les interruptions de service.

Etude de cas : Netflix

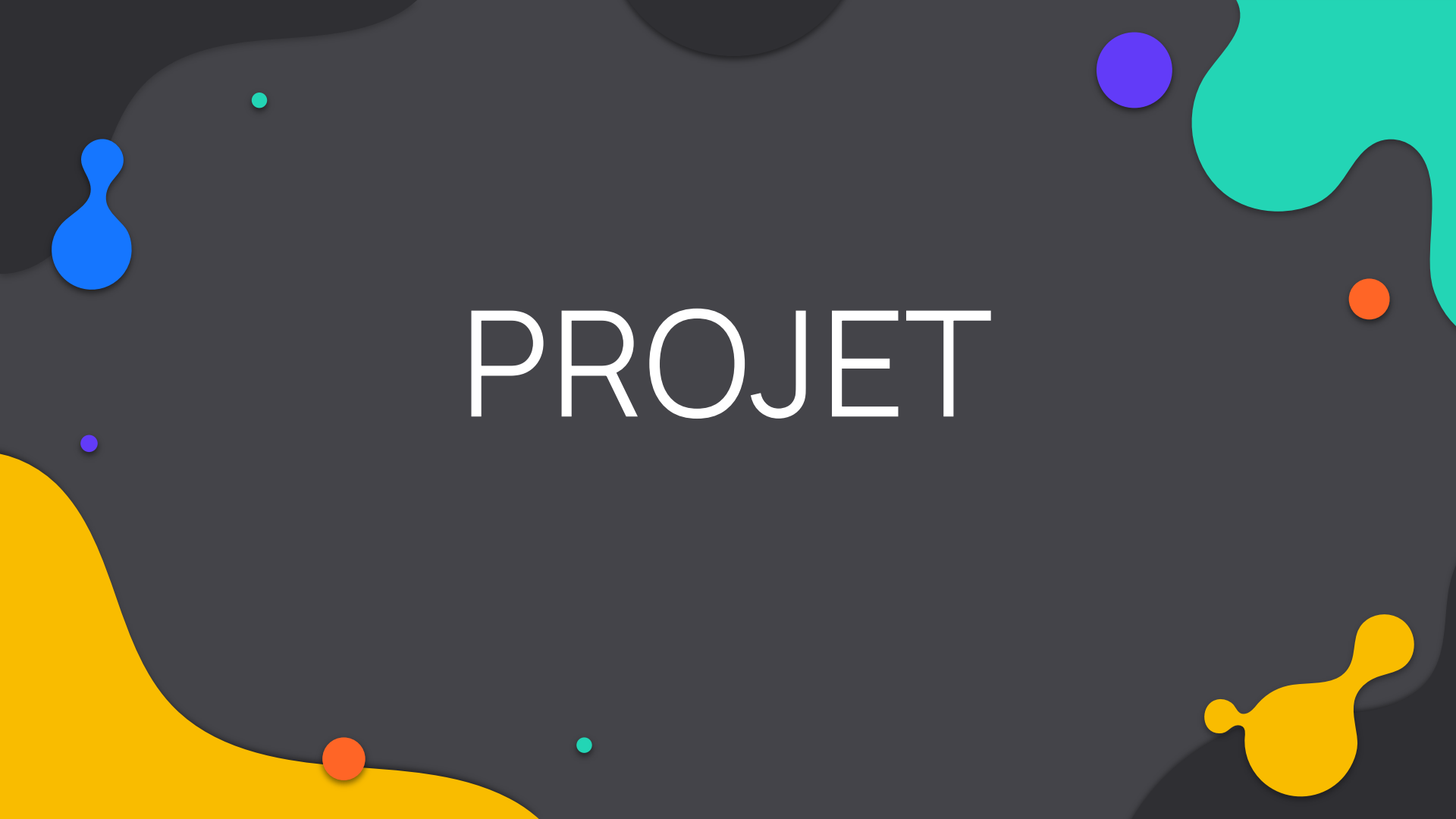
Résultats :

- Meilleure scalabilité :
Les services peuvent être indépendamment mis à l'échelle selon la demande.
- Augmentation de la résilience :
Les défaillances d'un service spécifique ont moins d'impact sur l'ensemble du système.
- Innovation accélérée :
Les équipes peuvent développer et déployer de nouvelles fonctionnalités plus rapidement et de manière plus autonome.

- 
- L'étude de cas de Netflix montre comment les microservices peuvent transformer la capacité d'une entreprise à innover tout en gérant de grandes échelles d'opération.

Cela met en évidence l'importance de la culture organisationnelle et de l'adoption des bonnes technologies pour soutenir une architecture de microservices.



The background is a dark gray with several colorful organic shapes and dots. In the top right, there is a large teal shape and a purple circle. In the bottom right, there is a yellow shape. In the bottom left, there is a large yellow shape. A blue shape is on the left side. Several small dots in teal, orange, and purple are scattered across the background.

PROJET